Jac and WeBridge: measured SQL fusion and database round trips

Two complete experiments · eight configurations · fixed nonempty IC5 input

Experiment 1: request latency at 0, 2 and 10 ms injected response delay.

Experiment 2: the exact SQL operations and serialized command/response cycles for every configuration.

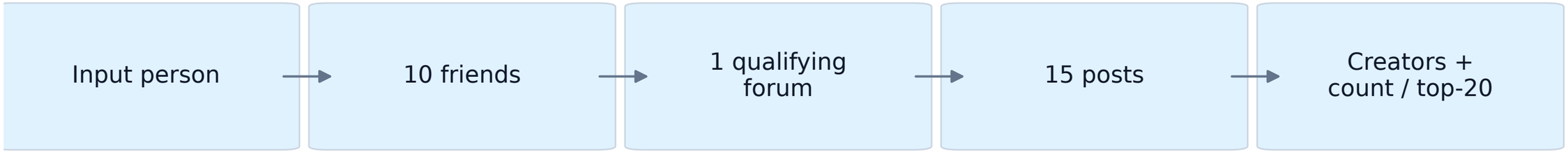
Jac emits fewer business queries through batching and relational aggregation. WeBridge full/pruned puts the original business-query sequence inside one CALL. Set-oriented Java already expresses the graph calculation as one SQL query.

960 main timing requests + 240 reverse-order requests; 30 measured protocol profiles.

Evidence: eight-arm-verification.json; webridge-operation-map.json; Jac roundtrip-profile/results/operation-map.json

One input and matched logical graph data

460,984 anchors · 300,901 edges · exact-result checks in every timing request



personId = 8796093030404; minDate = 1347527525888

Expected: forum 2061584375281, “Album 7 of Jie Xie”, postCount 0.

Jac uses PostgreSQL 18.6 and runtime mirror/adjacency structures. Java uses MySQL 5.7.44 over generic anchors. Cross-system timing compares these complete implementations.

Evidence: alignment/small-diagnostic/input.json; jac-comparison/fixture/load-receipt.json; correctness gates

Experiment 1 — complete latency matrix

Median execution time (ms) · 10 warmup + 30 measured requests per cell

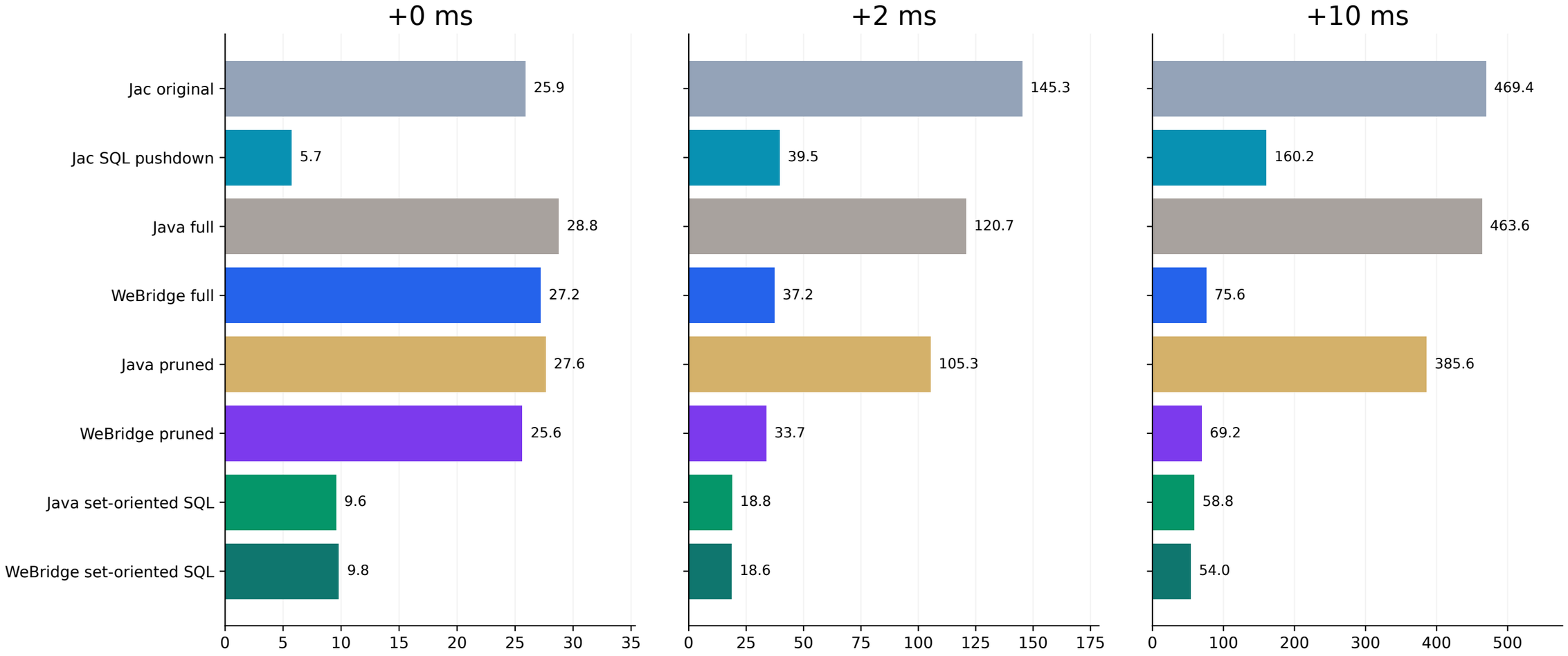
Configuration	0 ms	+2 ms	+10 ms	SQL / sync cycles
Jac original	25.89	145.26	469.42	38
Jac SQL pushdown	5.73	39.53	160.23	14
Java full	28.75	120.74	463.64	38
WeBridge full	27.20	37.25	75.63	4
Java pruned	27.64	105.27	385.60	31
WeBridge pruned	25.62	33.70	69.23	4
Java set-oriented SQL	9.56	18.79	58.82	4
WeBridge set-oriented SQL	9.81	18.61	53.96	4

All settings use the same delay-proxy path. Jac: server-total + commit timer. Java: JDBC request timer. Both exclude response JSON serialization.

Evidence: eight-arm-verification.json and raw per-request rows; calibrated SELECT 1 receipts

Experiment 1 — latency under added delay

Each panel has its own millisecond scale; values come from the complete matrix.



Evidence: eight-arm-verification.json · raw timing is separate from instrumented profiles

Set-oriented Java through actual WeBridge

Original recording → original Zero JVM → synthesis → generated procedure → execution

Delay	Direct, first	CALL, second	Direct, second	CALL, first
+0 ms	9.56	9.81	9.57	10.19
+2 ms	18.79	18.61	18.51	18.83
+10 ms	58.82	53.96	55.40	57.73

Both paths make four command/response cycles. At 10 ms the ordering reverses when phase order is reversed. These measurements show no consistent WeBridge speedup for the already set-oriented query.

All 20 original parameterized stored-procedure result gates pass. Both timing orders pass all 480 requests, with one CALL only in the optimized arm.

Evidence: alignment/idiomatic/WEBRIDGE.md; webridge-independent-verification.json; generated SQL SHA256

Experiment 2 — complete measured synchronization matrix

“Round trip” here means a serialized request followed by a complete database response.

Configuration	Sync cycles	Business / runtime reads	Outer txn commands	Replicates
Jac original	38	35 separate reads	3	6
Jac SQL pushdown	14	11 separate reads	3	6
Java full	38	35 separate reads	3	3
WeBridge full	4	35 inside one CALL	3	3
Java pruned	31	28 separate reads	3	3
WeBridge pruned	4	28 inside one CALL	3	3
Java set-oriented SQL	4	1 aggregate SELECT	3	3
WeBridge set-oriented SQL	4	1 aggregate inside CALL	3	3

Evidence: MySQL protocol-analysis.json (three postwarm requests per arm); PG flight-summary.json (six per variant)

Jac: which operations are combined, and which remain?

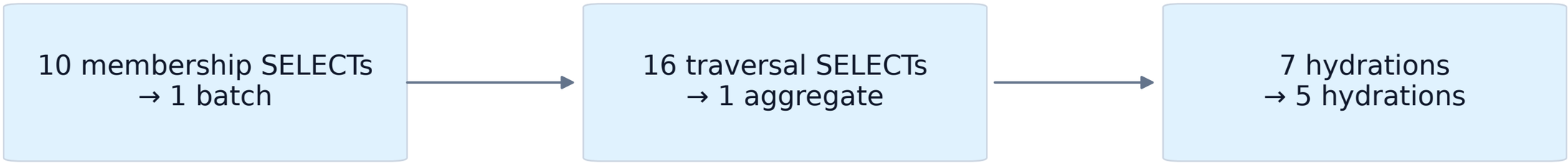
Exact warm PostgreSQL statement sequence; the original and final variants use the same walker.

Operation	Original	Final	Measured emitted form
Transaction start / setting / commit	3	3	Separate
Anchor hydration	7	5	Root, person, two friend batches, forum remain
Knows traversals	2	2	Outgoing and incoming remain separate
HasMember traversals	10	1	One batch over ten friends
Standalone type-ancestry probes	0	2	HasMember and Forum probes remain
ContainerOf + HasCreator	1 + 15	0	Combined into aggregate query below
Count / ordering / LIMIT SQL	0	1	Member-driven relational aggregate
Total SQL / synchronization cycles	38	14	All observed in each profile request

Evidence: Jac roundtrip-profile/results/operation-map.json and sql-catalog.json; protocol + backend SQL logs

Jac: account for every removed and retained cycle

The observed reduction is exactly 38 → 14.



38 − 9 − 15 − 2 + 2 type probes = 14

The two Knows traversals remain separate. Five anchor-loading queries and both type probes remain separate. The single forum is hydrated before the fused count query in this measured sequence.

WeBridge full/pruned moves their corresponding original queries into CALL; its executed category counts are on the next page.

Evidence: operation-map.json; named prepared statement catalog; bound frontier counts and result command tags

WeBridge: exact business operations executed inside CALL

Direct table SELECTs and executed cursor materializations are matched to generated procedure SQL.

Operation	WeBridge full	WeBridge pruned	WeBridge set-oriented
Hydration SELECTs	7	0	0
Knows traversals	2	2	part of aggregate
HasMember traversals	10	10	part of aggregate
ContainerOf traversals	1	1	part of aggregate
HasCreator traversals	15	15	part of aggregate
Set-oriented aggregate	0	0	1
Total business table operations	35	28	1
Outer transaction commands	3	3	3

Evidence: webridge-operation-map.json; actual P_S child events, cursor IDs and generated declarations

What each transformation achieves in this request

Compare the logical query boundary separately from the client/server boundary.

Comparison	Jac relational combination	Queries / boundaries still separate
Jac vs WeBridge full	Membership 10→1; post/creator 16→1; hydration 7→5	Jac retains 14 client cycles; WeBridge full uses 4
Jac vs WeBridge pruned	Jac batches membership and fuses count/order/LIMIT; pruned CALL retains 28 traversal operations	Pruned removes all hydration; Jac retains 5 hydration + 2 type probes
Jac vs set-oriented Java / CALL	Both have database-side aggregation and top-20	Set-oriented SQL also includes friend and membership joins in its single query; Jac emits them separately

Within the measured capture region, WeBridge full/pruned captures every original business SELECT. START, SET and COMMIT remain outside. The set-oriented variant captures its one aggregate SELECT.

Evidence: SQL operation maps and actual command sequences for all eight configurations

Protocol frames and return traffic are different from round trips

Actual MySQL frame decoding honors negotiated CLIENT_DEPRECATE_EOF.

MySQL configuration	Sync cycles	Response frames	Result sets	Response bytes
Java full	38	276	35	29,286
WeBridge full	4	970	36	68,300
Java pruned	31	176	28	6,798
WeBridge pruned	4	608	29	30,547
Java set-oriented SQL	4	9	1	220
WeBridge set-oriented SQL	4	17	2	495

Every next command follows the prior complete response in the trace. One CALL can return many protocol frames and result sets. For PostgreSQL, each stable execution is one Bind/Describe/Execute/Sync → ReadyForQuery cycle; the next Bind follows Ready delivery.

The PostgreSQL baseline uses 39 server socket chunks for 38 cycles; final uses 14 chunks for 14 cycles. Socket chunk count is recorded separately.

Evidence: mysql-results/protocol-analysis.json; mysql-idiomatic-results/protocol-analysis.json; parser regression checks

Inside the stored procedures: original work and generated work

Directly nested Performance Schema events for a complete observed CALL.

Observed event category	Full	Pruned	Set-oriented
Business table SELECTs	25	24	1
Business cursor materializations	10	4	0
Result / parameter metadata SELECTs	11	5	1
Variable / control instrumentation events	2,447	1,510	12

Business totals are 25+10=35 and 24+4=28. Cursor opens materialize original business queries.

Result/parameter SELECTs and variable/control events are generated procedure work.

Control-event totals are stored-program instrumentation counts. The set-oriented procedure executes its original aggregate plus one metadata SELECT and still requires four outer command/response cycles.

Evidence: webridge-operation-map.json · parent CALL timers include children; counts here use direct children only

Evidence and reproduction

Both experiments have a complete eight-configuration result matrix.

Latency	eight-arm-verification.json: 24 main cells, 960 checked requests; 240 additional reverse-order requests.
SQL / protocol	PG operation-map + flight-summary: 12 measured requests; MySQL protocol analyses: 18 measured requests.
Actual WeBridge	Generated procedure SHA256, original capture/replay/synthesis/install receipts, all20 parameter gates.
Independent checks	verify_eight_arms.py; audit_operation_map.py; test_mysql_parser.py; Jac roundtrip-profile/analyze.py.
Raw evidence	Per-request timings, payload-free protocol metadata, complete SQL catalog, P_S events and cleanup receipts.